Rack::Attack Rate Limiting Anleitung

Schritt-für-Schritt-Anleitung zur Konfiguration und zum Testen in Ruby on Rails

Damit `Rack::Attack` in einer Rails-Entwicklungsumgebung (Development) korrekt funktioniert, müssen Sie das **Caching aktivieren** und sicherstellen, dass Rails diese Middleware nicht überspringt.

Hier ist die konkrete Schritt-für-Schritt-Anleitung zur Konfiguration, Überprüfung und zum erfolgreichen Testen:

1 Schritt 1: Caching in der Entwicklungsumgebung aktivieren

Standardmäßig ist das Caching in der Rails-Dev-Umgebung **deaktiviert**. `Rack::Attack` benötigt zwingend einen Cache-Store (wie `MemoryStore` oder `Redis`), um die Anzahl der Anfragen zu zählen.

Führen Sie folgenden Befehl im Terminal Ihres Rails-Projektverzeichnisses aus:

```
bin/rails dev:cache
```

> *Wenn die Meldung `Development mode is now cached.` erscheint, war der Befehl erfolgreich.*

2 Schritt 2: Datei `config/initializers/rack_attack.rb` aktualisieren

Erstellen oder ersetzen Sie den gesamten Inhalt der Datei `config/initializers/rack_attack.rb`:

```
class Rack::Attack
  # 1. Cache Store für Rack::Attack deklarieren (MemoryStore verwenden, falls kein Redis genu
  Rack::Attack.cache.store = ActiveSupport::Cache::MemoryStore.new

  # 2. GraphQL-Route definieren
  GRAPHQL_PATH = "/graphql"

  # 3. Limit: 5 Anfragen / 1 Minute / IP
  throttle("graphql/ip/minute", limit: 5, period: 1.minute) do |request|
    # Überprüfe sowohl POST-Anfragen an /graphql als auch an /graphiql
    if request.post? && (request.path == "/graphql" || request.path == "/graphiql")
      request.ip
    end
  end

  # 4. JSON-Antwort bei Überschreitung des Rate Limits zurückgeben
  self.throttled_responder = lambda do |request|
    match_data = request.env["rack.attack.match_data"] || {}
    period = match_data[:period]
    now = match_data[:epoch_time]

    retry_after = period - (now % period) if period && now

    headers = {
      "Content-Type" => "application/json"
    }
    headers["Retry-After"] = retry_after.to_s if retry_after

    body = {
      error: "Too many requests",
      code: "RATE_LIMITED"
    }

    [429, headers, [body.to_json]]
  end
end
```

3 Schritt 3: Sicherstellen, dass die Middleware in Rails geladen wird

Öffnen Sie die Datei `config/application.rb` und fügen Sie die folgende Konfigurationszeile innerhalb von `class Application < Rails::Application` ein:

```
module YourApp
  class Application < Rails::Application
    # Rack::Attack zur Middleware-Pipeline von Rails hinzufügen
    config.middleware.use Rack::Attack
  end
end
```

4 Schritt 4: Rails Server neu starten

Nach Änderungen an den Konfigurationen in `initializers` und `application.rb` müssen Sie den Server **zwingend neu starten**:

```
# Drücken Sie Strg + C, um den aktuellen Server zu stoppen, und führen Sie dann aus:
bin/rails server
```

5 Schritt 5: Rate Limiting testen

Öffnen Sie ein neues Terminal und führen Sie den folgenden `curl`-Befehl aus, um **6 aufeinanderfolgende Anfragen** an den Server zu senden:

```
for i in {1..6}; do echo -n "Request $i: "; curl -X POST http://127.0.0.1:3000/graphql -H "Content-Type: application/json" -d '{"query": "{ __typename }"}' -s -o /dev/null -w "%{http_code}
"; done
```

Erwartetes Ergebnis:

```
Request 1: 200
Request 2: 200
Request 3: 200
Request 4: 200
Request 5: 200
Request 6: 429
```

Die 6. Anfrage gibt den HTTP-Code `429` zurück, was bedeutet, dass `Rack::Attack` die Anfrage erfolgreich blockiert hat.